

← All articles

AI TRENDS / 12 MINUTES READ

Our recommendations for creating API documentation (with examples)

JUNE 25, 2025



Emma Adler

Contributor @ Hackmamba

SHARE THIS ARTICLE



★ SUMMARY

This comprehensive guide provides actionable recommendations for creating API documentation that converts developers, covering essential components from API references to error handling with real-world examples from Stripe, Twilio, and Pinecone. The article emphasizes designing documentation for both human developers and LLMs, offering practical strategies for structure, code samples, and maintenance that drive API adoption and reduce support burden.

Developers stick with APIs they can understand quickly, try without friction, and trust to behave as expected. That experience starts with clear, structured documentation written for humans and large language models (LLMs).

Today, your docs power more than onboarding. LLMs are parsing your endpoints, generating code from your examples, and answering support questions based on your formatting and structure. How you write your documentation shapes how developers interact with your API.

In this guide, you'll learn how to create documentation that's easy to navigate, rich in examples, and designed to shorten time to first call.

We'll break down the essential components, share standout examples, and outline the practices we've seen work best across thousands of Mintlify-powered docs, from fast-moving startups to teams running complex, versioned APIs.

What is API documentation?

API documentation is a set of instructions that explain how to interact with an API. That includes available endpoints, methods, resources, common requests and response formats, authentication methods, error codes, and example requests.

However, that definition alone undersells what API documentation truly accomplishes.

High-quality API documentation is what enables developers, often on their first touchpoint with your product, to build something meaningful with it. It's one of the most critical components of API documentation, especially in an API-first world,



where effective API documentation drives adoption.

The cost of bad API documentation

When you have bad API documentation, you risk a poor developer experience and also lose momentum across your entire product surface:

- Your support team becomes the de facto documentation team.
- API adoption stalls. Engineers trialing your API give up before reaching the “aha” moment, especially if they can't find relevant examples and code samples or understand the API's functionality.
- Trust breaks. Incomplete or inconsistent docs signal that the product itself might be unreliable.
- Internal usage suffers. Even your own teams can't confidently work with the API if the docs don't reflect current behavior.

Most of the time, you don't know what your documentation is costing you until it's too late. Because not all developers will log a complaint, many will immediately switch to your competitors with better docs and a smoother experience.

Common mistakes when creating documentation

While some teams don't care about creating great API documentation, others genuinely want to, but somehow still end up with subpar documentation due to some of these mistakes:

1. Context mismatch

A common failure is writing docs with the wrong audience in mind. Internal docs can afford to assume shared knowledge. External docs can't. However, many teams blur the line, leaving public-facing APIs documented as if only internal teams will ever use them.

Assuming users are familiar with what you know, such as naming conventions, authentication flows, and edge cases, makes it difficult for them to use your API documentation.

2. Believing structure can be automated



Automated API reference docs are a good starting point—but they're only that: a starting point. They do not provide structure or tackle use cases, edge cases, or

workflows. They tell you what the API's functionality is, not how or why to use it.

And while LLMs can help fill in gaps, they're not a substitute for editorial judgment.

3. Locking documentation behind authentication walls

Unless there's a strong security or privacy reason, requiring developers to log in to access your API documentation is unnecessary. Developers evaluating your API want to self-serve, and that starts with reading the docs. If they can't do that without an account, many simply won't. This, in turn, impacts the adoption of APIs.

4. Outdated content

APIs change over time, but many teams don't have systems in place to keep documentation up to date. This often leads to cases where you have code samples or example responses that no longer work, parameters that no longer exist, or changelogs that don't reflect real changes in your API docs.

What are the most common types of API documentation?

Here are the core components of API documentation that every developer-facing API should include, each serving a distinct purpose in the integration journey:

API reference

This is the backbone of your docs; it describes and demonstrates. A great API reference shows the complete request/response cycles, so developers don't have to guess what the API returns.

It includes detailed information about every endpoint operation and resource:

- Paths and methods (GET, POST, etc.)
- Parameters (query, path, body)
- Request and response schemas
- Status codes and error messages

While the reference docs tell you what's possible, guides show you how to get it done. This is where you guided developers through real-world tasks and authentic use cases that mirror what they're likely building.

ANTHROPIC English Search... Research Login Sign up

Welcome Developer Guide **API Guide** Claude Code Model Context Protocol (MCP) Resources Release Notes

API reference

- Messages
- Models
- Message Batches
 - Create a Message Batch** (POST)
 - Retrieve a Message Batch (GET)
 - Retrieve Message Batch Results (GET)
 - List Message Batches (GET)
 - Cancel a Message Batch (POST)
 - Delete a Message Batch (DEL)
- Files
- Admin API
- Experimental APIs
- Text Completions (Legacy)
- SDKs
 - Client SDKs

MESSAGE BATCHES

Create a Message Batch

Send a batch of Message creation requests.

The Message Batches API can be used to process multiple Messages API requests at once. Once a Message Batch is created, it begins processing immediately. Batches can take up to 24 hours to complete.

Learn more about the Message Batches API in our [user guide](#)

POST /v1/messages/batches

Feature Support

The Message Batches API supports the following models: Claude Haiku 3, Claude Opus 3, Claude Sonnet 3.5, Claude Sonnet 3.5 v2, Claude Sonnet 3.7, Claude Sonnet 4, and Claude Opus 4. All features available in the Messages API, including beta features, are available through the Message Batches API.

Batches may contain up to 100,000 requests and be up to 256 MB in total size.

```
curl https://api.anthropic.com/v1/messages/batches \
  --header "x-api-key: $ANTHROPIC_API_KEY" \
  --header "anthropic-version: 2023-06-01" \
  --header "content-type: application/json" \
  --data '{
    "requests": [
      {
        "custom_id": "my-first-request",
        "params": {
          "model": "claude-3.5-sonnet-20250219"
        }
      }
    ]
  }'
```

```
{
  "archived_at": "2024-08-20T18:37:24.100435Z",
  "cancel_initiated_at": "2024-08-20T18:37:24.100435Z",
  "created_at": "2024-08-20T18:37:24.100435Z",
  "ended_at": "2024-08-20T18:37:24.100435Z",
  "expires_at": "2024-08-20T18:37:24.100435Z",
  "id": "msgbatch_0132va2CMHLNnXjN3JkQJ2EF",
  "processing_status": "in_progress",
  "request_counts": {
    "canceled": 10,
    "errored": 30
  }
}
```

Anthropic linking to a user guide from the API reference

Changelogs

Your changelog should be visible, readable, and treated as an important part of the documentation.

This documentation should:

- Be visible within your documentation
- Call out breaking changes, deprecations, new endpoints, and SDK updates
- Include dates and links to relevant sections or PRs
- Be treated like product announcements

* At Mintlify, we treat changelogs as first-class documentation: timestamped, discoverable, and always linked to the relevant changes.

Authentication and authorization

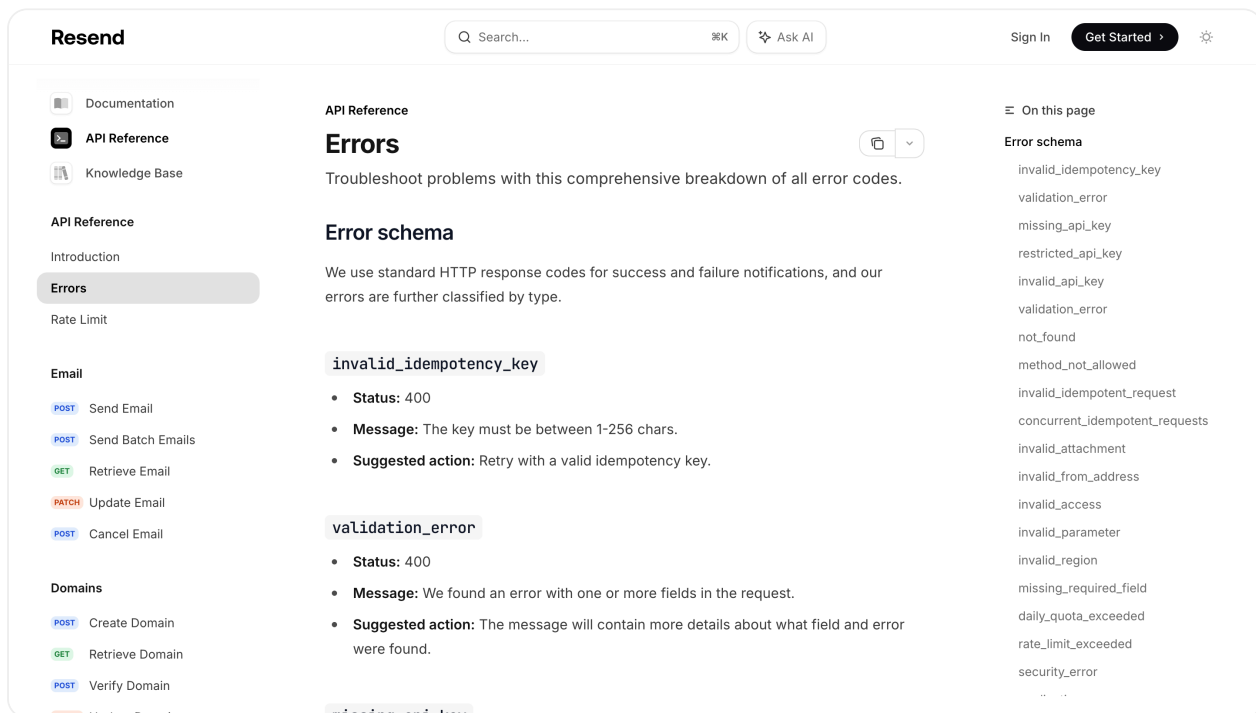

Authentication deserves its own section, as it shows developers have access to your API. This part of your docs should:

- Walk through the credential setup step by step
- Show where to include tokens (headers, query params, etc.)
- Include examples for curl, Postman, and at least one SDK (if you offer them)
- Explain token expiry, rate limits, and scopes clearly

If you support multiple auth methods (OAuth, API keys, JWTs), be explicit about which to use and when.

Error handling

Document every error code your API can return, with context, and offer actionable guidance on how to fix these errors. For example, what does a 403 mean in your system? What's the difference between a 400 and a 422?



Error handling in the Mintlify-powered Resend documentation

What good API documentation looks like (with real examples)

At Mintlify, we believe that your API documentation is part of the product. We design documentation to feel intuitive and helpful at every step of the developer journey. That means:

- Structuring docs around real workflows
- Making changelogs, error handling, and usage examples first-class citizens
- Building for both humans and LLMs

When documentation is designed this way, it helps developers move faster, troubleshoot less, and build with confidence.

Here's how that philosophy comes to life in practice:

Clear, structured, and purposeful

Stripe's documentation is often held up as the gold standard, and for good reason.

Every one of their API reference page includes:

- A descriptive title (not only POST /v1/charges)
- Live code samples in multiple programming languages
- Example requests and example responses with actual data
- Clear, skimmable descriptions and usage notes

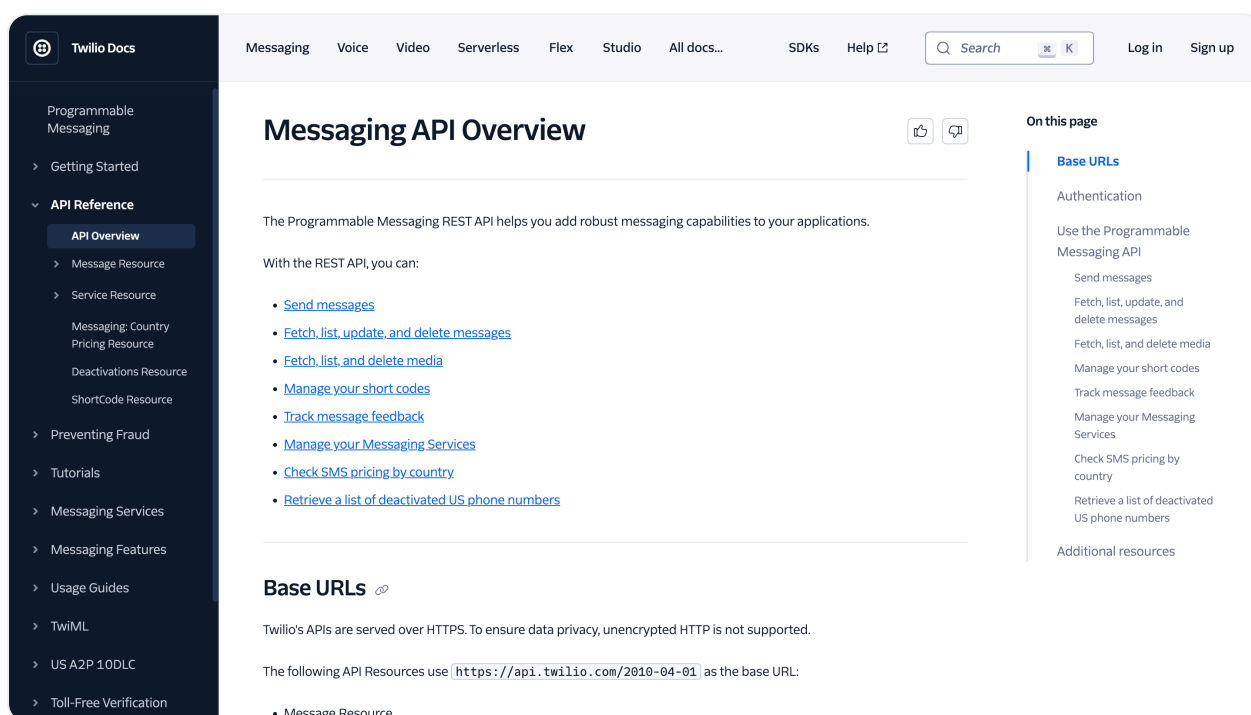
The screenshot displays the Stripe API documentation interface. On the left, a sidebar contains a search bar and a navigation menu with categories like 'Introduction', 'Core Resources', and 'Authentication'. The main content area is titled 'API Reference' and includes sections for 'Just getting started?', 'Not a developer?', and 'Authentication'. The 'Just getting started?' section provides a 'BASE URL' of `https://api.stripe.com` and lists 'CLIENT LIBRARIES' for various programming languages. The 'Authentication' section explains how to use API keys. The bottom of the page features a 'mintlify' logo and a 'GLOBAL API KEY' section.

Stripe documentation API reference

With this structure, you're never left wondering "What does this do?" or "How do I use it?" Stripe's documentation is organized around real developer goals, not just a list of endpoints, methods, resources, and authentication protocols.

Real-life use cases

Twilio's guides are structured around examples of common requests like "Send messages," "Record a call," or "Verify a WhatsApp number." With this, you're never just staring at a reference; you're building things.



Twilio documentation showing how the API reference is structured for real-life use cases

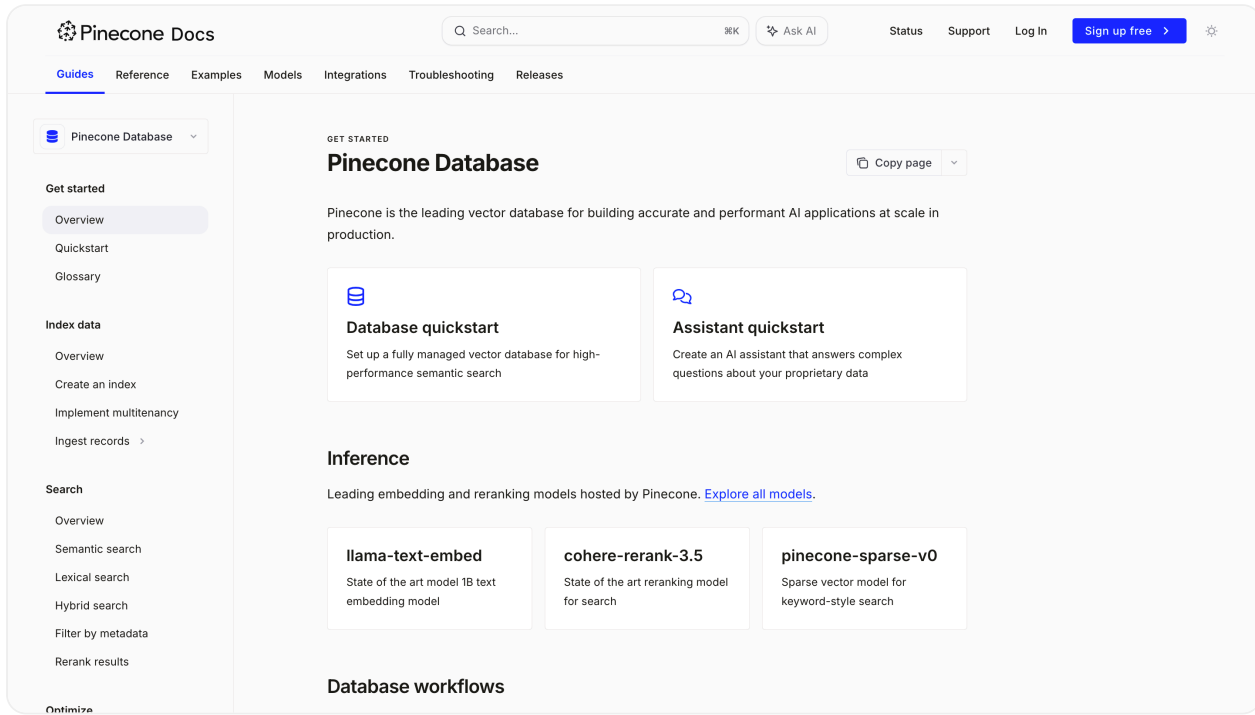
Similarly, Next.js Commerce goes beyond a "Hello World" and shows developers how to structure production-ready applications. This approach doubles as a powerful marketing content and onboarding material.

Great developer experience (DX)

Pinecone's documentation is another standout in terms of DX. It's clearly built by people and tools that understand that documentation facilitates cross-team collaboration. It features:



- Code blocks that switch based on the selected language
- Guides mapped to key workflows like indexing or querying
- A release notes section that's easy to find
- Lightweight, distraction-free design for fast readability



Pinecone documentation showing a smooth DX layout

These choices say to the developer: "We value your time."

Error handling

Stripe again sets the bar here. Their error documentation walks you through each type of error from validation, auth, and rate limiting, with:

- Structured error codes and meanings
- Retry guidance and best practices for creating great apps
- Examples and code samples for handling errors in various client languages



stripeDOCS

Q Search

Ask AI

Create account

Sign in

Get started

Payments

Revenue

Platforms and marketplaces

Money management

Developer tools

APIs & SDKs

Help

API keys

Stripe-Context header

Daily changelog

Rate limits

Automated testing

Metadata

Expanding responses

Pagination

Domains and IP addresses

Search

Localization

Error handling

Advanced error handling

Error codes

Testing

Workbench

Event Destinations

Workflows

Stripe CLI

Stripe Shell

Developers Dashboard

United States

English (United States)

Home / Developer tools / API

Error handling

Catch and respond to declines, invalid data, network problems, and more.

Ruby

Python

PHP

Java

Node.js

Go

.NET

Stripe offers many kinds of errors. They can reflect external events, like declined payments and network interruptions, or code problems, like invalid API calls.

To handle errors, use some or all of the techniques in the table below. No matter what technique you use, you can follow up with our recommended responses for each error type.

TECHNIQUE	PURPOSE	WHEN NEEDED
Catch exceptions	Recover when an API call can't continue	Always
Monitor webhooks	React to notifications from Stripe	Sometimes
Get stored information about failures	Investigate past problems and support other techniques	Sometimes

Catch exceptions

ON THIS PAGE

Catch exceptions

Monitor webhooks

Get stored information about failures

Types of error and responses

Payment errors

Payment blocked

Payment declined

Other payment errors

Invalid request errors

Connection errors

API errors

Authentication errors

Idempotency errors

Permission errors

Rate limit errors

Signature verification errors

Error handling in Stripe documentation

This type of content is critical in an API-first product and helps ensure fewer support tickets.

Updated and maintained changelog

Perplexity changelog is treated with intentionality; you can tell by how it's structured and presented. Every product update is announced with care, context, and simplicity. This results in users feeling informed, rather than beingblindsided.

sonarby perplexity

Q Search docs

Ask AI

Playground

Home

Models

Guides

API Reference

Changelog

System Status

FAQ

Roadmap

Discussions

Changelog

Overview

Changelog

Changelog

New: Academic Filter for Scholarly Research

June 2025

New: Reasoning Effort Parameter for Sonar Deep Research

May 2025

New: Asynchronous API for Sonar Deep Research

May 2025

Enhanced API Responses with Search Results

May 2025

New API Portal for Organization Management

April 2025

New: Location filtering in search

April 2025

Image uploads now available for all users!

April 2025

New: Date range filtering in search

April 2025



Changelog in Sonar by Perplexity

As their product evolves, they publish documentation alongside the changelog, highlighting new features, flagging deprecated endpoints and outdated documentation, and explaining version changes in plain language.

How to make your API documentation good, too

Every one of these API documentation examples, from Stripe to Pinecone, made it to the list because they were intentional about every detail. Writing high-quality API documentation requires more than technical accuracy. It demands empathy, consistency, and ongoing investment.

At Mintlify, we've worked with companies like Perplexity, Vercel, Resend, and Captions to build API documentation that accelerates onboarding, reduces support burden, and helps teams move faster. Here are the key practices we've seen make the biggest difference:

Assign clear ownership of documentation

One of the most common reasons we've seen as to why API documentation fails is that nobody owns it. If it's left to everyone, then it's no one's job. Assigning ownership helps drive accountability and long-term consistency.

Someone on your team, ideally a technical writer, DevRel, or dedicated doc maintainer, needs to be responsible for:

- Keeping docs current as the API evolves
- Reviewing developer feedback
- Collaborating with support, product, and engineering to close doc gaps

Start with the developer journey

Don't stop at documenting endpoints alone; map out the developer's path. What does success look like for a first-time user? What are developers using your product for, and how fast can they start building with the API?



Work backward from that moment. Build your guides, API reference sections, and code samples to lead developers toward it with minimal friction.

Pair guides with reference docs

Yes, auto-generating OpenAPI specs are important, but they're just one layer of the solution. Pair them with opinionated guides that reflect real use cases. Perplexity uses the [Mintlify API playground](#) to layer interactive playgrounds over their references, turning spec pages into executable experiences.

The screenshot displays the 'Perplexity API' documentation on the 'sonar by perplexity' website. The 'API Reference' tab is active, showing the 'Chat Completions' endpoint (POST /chat/completions). The endpoint description states: 'Generates a model's response for the given chat conversation.' A 'Try it' button is present. Below the endpoint, 'Authorizations' are listed as 'Bearer authentication header of the form Bearer <token>, where <token> is your auth token.' The 'Body' is specified as 'application/json'. A 'model' parameter is listed as 'string' and 'required', with a description: 'The name of the model that will complete your prompt. Refer to [Supported Models](#) to find all the models offered.' An example is given: 'sonar'. On the right, a 'Playground' section shows a cURL request and a 200 JSON response.

```
curl --request POST \
  --url https://api.perplexity.ai/chat/completions \
  --header 'Authorization: Bearer <token>' \
  --header 'Content-Type: application/json' \
  --data '{
    "model": "sonar",
    "messages": [
      {
        "role": "system",
        "content": "Be precise and concise."
      },
      {
        "role": "user",
        "content": "What is the capital of France?"
      }
    ]
  }'
```

```
{
  "id": "<string>",
  "model": "<string>",
  "created": 123,
  "usage": {
    "prompt_tokens": 123,
    "completion_tokens": 123,
    "total_tokens": 123,
    "search_context_size": "<string>",
    "citation_tokens": 123,
    "num_search_queries": 123,
    "recognition_tokens": 123
  }
}
```

Perplexity documentatiin playground feature powered by Mintity

For example, instead of only listing the /payments endpoint, include a "How to accept a payment in your app" guide that walks developers through setting up authentication, making API calls, and handling the response.

Use real, copy-paste-ready code samples

Great documentation shows more than it tells. That means:

- Full working code blocks that developers can copy and run
- Expected responses, error examples, and edge cases



Resend uses Minlify's [tabs](#) to display code examples in multiple programming languages, alongside their corresponding responses. This makes it easy for

developers to copy, paste, and follow along regardless of the language they're using.

The screenshot shows the Resend API documentation page for the 'Send Email' endpoint. The page layout includes a sidebar on the left with navigation links for 'Documentation', 'API Reference', and 'Knowledge Base'. Under 'API Reference', there are links for 'Introduction', 'Errors', 'Rate Limit', and 'Email'. The 'Email' section is expanded, showing a list of endpoints: 'Send Email' (POST), 'Send Batch Emails' (POST), 'Retrieve Email' (GET), 'Update Email' (PATCH), and 'Cancel Email' (POST). The 'Send Email' endpoint is selected, showing its details. The endpoint is a POST request to '/emails'. The 'Body Parameters' section lists the following fields: 'from' (string, required), 'to' (string | string[], required), 'subject' (string, required), and 'bcc' (string | string[]). The 'Response' section shows a JSON object with an 'id' field. On the right side of the page, there is a code switcher for different programming languages: Node.js, PHP, Python, Ruby, Go, Rust, Java, .NET, and cURL. The PHP code is currently selected and displayed in a light blue box. The code is as follows:

```
import { Resend } from 'resend';

const resend = new Resend('re_XXXXXXXXXX');

await resend.emails.send({
  from: 'Acme <onboarding@resend.dev>',
  to: ['delivered@resend.dev'],
  subject: 'hello world',
  html: '<p>it works!</p>',
});
```

The 'Response' section shows a JSON object with an 'id' field:

```
{
  "id": "49a3999c-8ce1-4ea6-ab68-afcd6dc2e794"
}
```

Resend documentation showing the use of code switchers

Design for skim-ability and search

Developers come to your documentation with a specific task in mind, not to read everything. A great documentation experience is built to be scanned, searched, and navigated with minimal friction.

Mintlify provides configurations and internal and external search optimization and components to help you achieve that easily. If someone finds your documentation on Google, they should be able to scan and act within seconds.

Keep changelogs front and center

As your product evolves, developers need to understand what has changed, what is deprecated, and how to adapt. Mintlify advocates for treating changelogs as first-class content, instead of burying them in GitHub releases or fragmenting them across tools.

Use feedback loops



Add simple ways to collect feedback on every page of your documentation. That way, it's easy to see which parts of your API documentation need work. Cursor uses inline feedback components in Mintlify to surface where the docs need updates.

Automate checks and schedule regular audits.

Build a habit of checking your docs for outdated flows, deprecated endpoints, expired tokens, or authentication steps that no longer apply.

Create a fixed schedule and automate where possible. With Mintlify, you can check for broken links, outdated tokens, and grammar issues directly from your GitHub workflow. Use Mintlify to automatically generate APIs and detect changes in your OpenAPI spec; you can also set up scripts to flag stale content, especially for high-traffic or frequently updated pages.

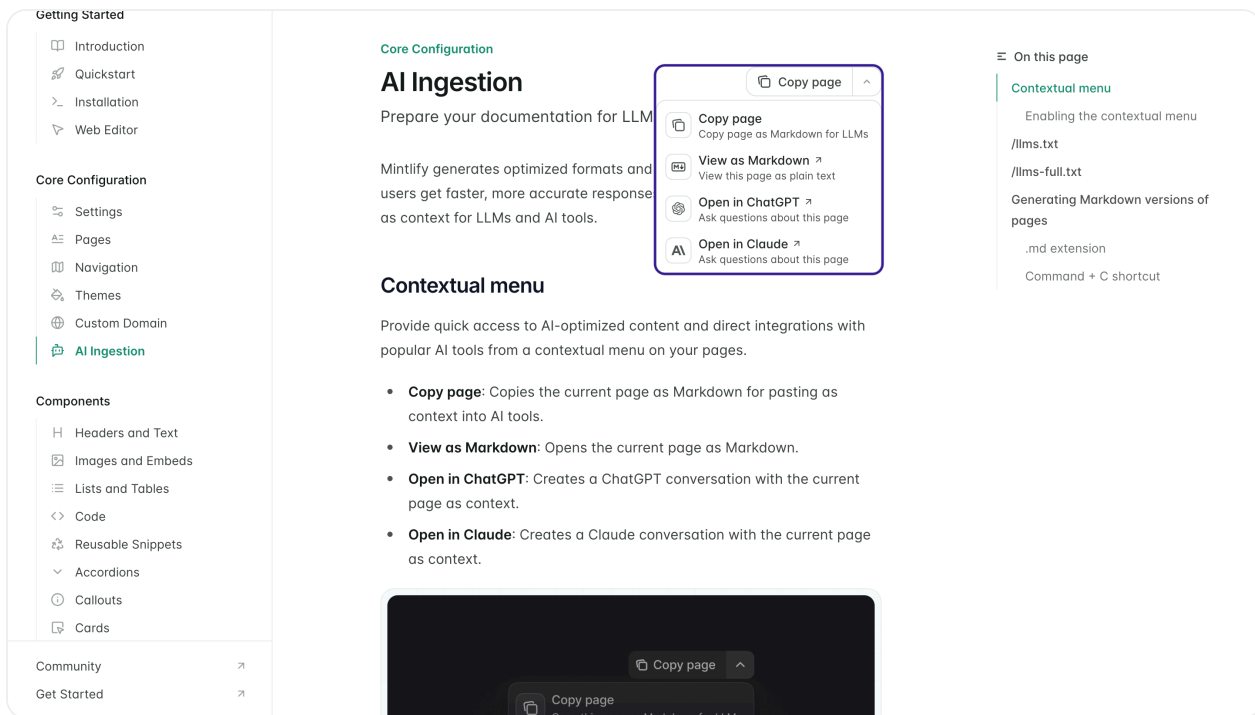
These are very important, especially for products with APIs that get updated regularly.

Write with both humans and LLMs in mind

To make your documentation easy for both humans and large language models (LLMs) to understand, it needs to be well-structured, consistently formatted, and filled with clear examples.

LLMs like GPT-4 and Claude can use your docs to generate code, write tests, or suggest integrations, helping users solve problems faster. That's why all Mintlify-powered docs are optimized for LLM consumption using our AI Ingestion feature.





Mintlify powered AI ingestion feature

Lean on your community

Your documentation can also be a feedback loop. Add lightweight feedback tools (like thumbs-up/down or "Was this helpful?" prompts) to identify unclear sections. Use that data to inform future updates.

What's next?

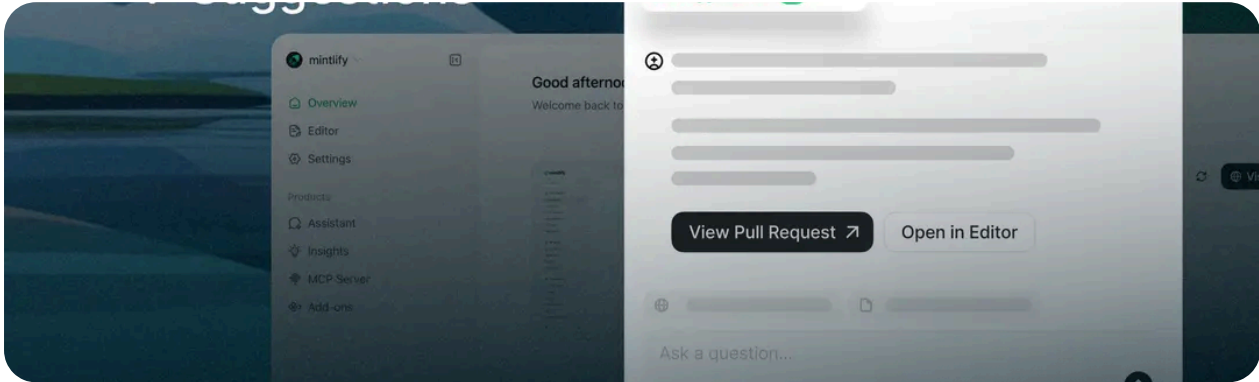
To improve your API documentation, start by identifying what's not working, whether that's outdated content, unclear organization, or difficulty keeping docs aligned with your codebase. It's worth taking time to evaluate your current documentation workflow and developer experience. Consider where developers get stuck, what content is missing, and how easy it is for your team to maintain accuracy over time.

Mintlify is designed to support this process with built-in support for Git-based updates, structured changelogs, language-specific code samples, SEO, and page-level feedback, among other developer-friendly features.



Mintlify currently helps thousands of companies, from next-gen startups to enterprises, to build and maintain world-class documentation that serves both developers and LLMs.

More blog posts to read



Closing the loop between user questions and documentation

Learn how Mintlify now surfaces documentation updates not only from pull requests but also from real user conversations, giving teams a clearer and more complete signal of what needs...

January 7, 2026



2025: A Year in Review

Scaling to 8-figures ARR, onboarding 10,000+ companies, and evolving from a documentation platform into the infrastructure layer for how AI understands technical knowledge.

December 31, 2025





EXPLORE

Startups

Enterprise

Switch

OSS Program

RESOURCES

Customers

Blog

Pricing

Guides

Feature Requests

DOCUMENTATION

Getting Started

API Reference

Components

Changelog

COMPANY

Careers

Wall of Love

LEGAL

Privacy Policy

Responsible Disclosure

Terms Of Service

Security

DSR/DSAR

Backed By Enterprise-Grade Security



🟢 All systems normal

© 2026 Mintlify, Inc.

